

Lab exercises

Track 16.3 –  LangChain · 16-ai-tech-stack/tracks/langchain/lab/EXERCISES.md

Lab — LangChain

Work top to bottom. Every exercise has a **stated deliverable** and a **check**; if you can't produce the deliverable, you haven't finished. Everything runs offline — no API key, no network.

Run from `../code/` so `fakes.py` is importable:

```
cd ../code && /Users/srinip/ai-all/.venv/bin/python lcel_basics.py
```

1. Warm-up: read what you were given (45 min)

Run all three scripts, then answer from the code, not from memory:

- **1a.** In `lcel_basics.py` demo 2, the chain with an appended `RunnableLambda` streams exactly one chunk. Explain the mechanism in one sentence, then change the chain so it streams again *and still uppercases the output*.
- **1b.** In `fakes.py`, `FakeChatModel` implements only `_generate`, `_stream` and `_llm_type`. List five public methods it gets for free, and name the class that provides them.
- **1c.** Delete the `run_manager.on_llm_new_token(...)` call in `FakeChatModel._stream` and re-run `lcel_basics.py` demo 6. Which event count changes, and why does that matter for LangSmith?

Deliverable: three short answers + the working 1a chain.

2. Build a model from scratch (1h)

Write `EchoBudgetChatModel(BaseChatModel)` that:

- replies with the last human message reversed;
- raises `RuntimeError("budget exceeded")` after `max_calls` invocations;
- reports `usage_metadata` with a real word count.

Checks: - `model.batch(["a", "b", "c"])` works without you writing a `batch` method. -

`model.with_retry(retry_if_exception_type=(RuntimeError,), stop_after_attempt=2)` retries and then raises — prove it, and explain why retrying a budget error is the wrong policy. -

`(ChatPromptTemplate.from_template("{q}") | model | StrOutputParser()).invoke({"q": "hello"})` returns `"olleh"`.

3. The plumbing puzzle (45 min)

Build a single LCEL chain that takes `{"text": ...}` and returns a dict with **all four** keys: `text` (unchanged), `summary`, `critique`, and `word_count` — with `summary` and `critique` produced by two *concurrent* model calls.

Checks: - The chain contains exactly one `RunnableParallel` and one `RunnablePassthrough.assign`. - `chain.invoke(...)` returns all four keys. - Instrument `FakeChatModel.call_count` to prove the model was called exactly twice. - Draw the chain as an ASCII diagram (or `chain.get_graph().print_ascii()` after `pip install grandalf`) and paste it into your answer.

4. Harden a flaky pipeline (1h)

Using `FlakyChatModel`, build a ladder that: retries the primary model up to 3 times on `ConnectionError` only, then falls back to a cheap model, then falls back to a canned apology string.

Checks: - With `fail_times=2` the primary succeeds and neither fallback fires. - With `fail_times=99` you get the cheap model. - With both models failing you get the canned string and **no exception**. - Add a `TimeoutError` case and show that your `retry_if_exception_type` does **not** retry it. Then argue in two sentences whether it should.

5. Make the tool loop misbehave (1h)

Extend `run_tool_loop` in `tools_and_structured_output.py` to survive three more attacks:

- **5a.** The model emits the *same* tool call twice in one `AIMessage` with the same `id`. What breaks, and what should the loop do?
- **5b.** A tool hangs. Add a per-tool timeout that produces a `ToolMessage(status="error")` rather than blocking the request. (`concurrent.futures` is enough.)
- **5c.** The model never stops calling tools. Prove your `max_turns` guard fires and that the history it returns is still a valid conversation — i.e. every `tool_call_id` has a `ToolMessage`.

Deliverable: a `validate_history(messages) -> list[str]` function that returns a list of protocol violations, and a test showing it returns `[]` for all three cases above.

6. Structured output under pressure (45 min)

Take `IncidentReport` from `tools_and_structured_output.py`.

- **6a.** Write `extract_with_repair(model, text, max_attempts=3)` using `include_raw=True`: on a `ValidationError`, re-invoke with the pydantic error text appended to the prompt.
- **6b.** Script a model that fails twice and succeeds on the third attempt. Prove the repair loop returns a valid `IncidentReport` and report how many model calls it cost.

- **6c.** Add a field `estimated_cost_usd: float = Field(ge=0)` and script a model that returns a *string* "about \$4000". What does pydantic do, and what would you show the user?

Check: your repair loop never loops forever and never swallows the final error silently.

7. RAG that knows when to shut up (1.5h) — the capstone

Extend `mini_rag_chain.py`:

1. Wrap the retriever so chunks below a similarity threshold are dropped. Pick the threshold by *measuring* — build a tiny set of 5 in-corpus and 5 out-of-corpus questions and tabulate scores.
2. If nothing survives, short-circuit: return an abstention **without calling the model**.
3. Return a pydantic `Answer(answer: str | None, sources: list[str], abstained: bool)`.
4. Add three documents from the same team with overlapping content, and show a case where the top-2 chunks are near-duplicates. Fix it (dedupe by source, or `max_marginal_relevance_search`) and show the retrieved set improve.

Checks: - "What is our parental leave policy?" now returns `abstained=True` and calls the model zero times (assert on `FakeChatModel.call_count`). - All in-corpus questions still answer correctly. - A table of your 10 measured scores and the threshold you chose, with a sentence on the trade: what does raising it cost you?

8. Test it like production (1h)

Write `test_pipeline.py` with `pytest`:

- a test that every `tool_call_id` is answered (reuse §5's validator);
- a test that the RAG chain streams more than one chunk;
- a **golden trace** test: capture `astream_events` event-type counts for a fixed input, store them, and fail if a refactor changes them;
- a test that the whole suite runs with no network (run it with networking disabled if you can, or assert that no provider package is imported).

Check: the full suite runs in **under 2 seconds**. If it doesn't, something is doing real I/O.

9. The argument (30 min)

Take the pipeline you built in §7 and write two paragraphs:

- **For:** the three things LangChain gave you that you would otherwise have written.
- **Against:** the same pipeline written against the raw provider SDK — estimate the line count, and name what you lose.

Check: hand both paragraphs to a colleague. If they cannot tell which side you personally hold, you have written the honest version. Then state your actual position in one sentence.

10. Stretch: graduate the pipeline (2h)

Rebuild §7 as a `create_agent` from `langchain.agents`, with the retriever exposed as a **tool** rather than wired into a fixed chain (agentic RAG instead of 2-step RAG), an `InMemorySaver` checkpointer, and a `thread_id`.

Checks: - A follow-up question ("and what about the search service?") resolves using conversation state. - Compare against your LCEL version on: number of model calls, latency, and predictability. - Write down the criterion you would use to choose between them on a real project. Then read `../langgraph/` and see whether you still agree.